Sales / Lead Conversion Intelligence System

for Small and Medium Enterprises

Complete project documentation · phase by phase · with viva questions and answers

B.Tech Information Technology · Mini Project
October 2026

Python · FastAPI · scikit-learn · SQLite · React · Tailwind CSS · Twilio WhatsApp

Contents

1. Project overview
    1. The problem
    2. Objectives
    3. What was built
    4. Key terms in simple words
2. System architecture
    1. Architecture diagram
    2. Technology stack
    3. Project files
    4. The nine modules
3. Phase 1 – Dataset and machine learning model
4. Phase 2 – Backend and database
5. Phase 3 – Reading the customer's message
6. Phase 4 – Website dashboard
7. Phase 5 – Channels: WhatsApp, website and email
8. Phase 6 – The WhatsApp assistant
9. Phase 7 – Learning from results
10. Phase 8 – Testing
11. How to run the project
12. Demonstration script
13. Limitations and future work
14. Viva questions and answers
    1. Project and problem
    2. Data and machine learning
    3. Scoring and messages
    4. Backend, database and dashboard
    5. WhatsApp, website and email
    6. Testing, security and limitations
15. Glossary

1. Project overview

1.1 The problem

Small and Medium Enterprises (shops, coaching centres, car dealers, salons, bakeries) receive customer enquiries every day on WhatsApp, Instagram, their website and email. A **lead** is one such person who has shown interest. Most leads never buy; a few are ready to buy now.

A small business has no data team and little time. Enquiries are answered in the order they arrive, hot buyers wait while the team chats with people who are "just checking", and follow-ups are forgotten. Large companies solve this with expensive CRM tools and data scientists. Small businesses usually cannot afford either.

> **Problem statement**
>
> Build a low-cost system that collects enquiries from the channels a small business already uses, estimates how likely each lead is to buy, explains why, tells the team whom to contact first, and drafts the follow-up message, while keeping a person in control of what is sent.

1.2 Objectives

1. Collect leads automatically from WhatsApp, a website enquiry form and email, as well as manual entry.
2. Predict each lead's **chance of buying** with a machine learning model trained on real lead data.
3. Read what the customer wrote and adjust the chance for buying signals (price question, "book this week", "not interested").
4. Give every lead a priority (HIGH / MEDIUM / LOW) and a plain-English reason.
5. Answer common customer questions (price, specifications, availability, delivery, timings) automatically from the owner's product list.
6. Remind the team which leads need a follow-up and draft the message.
7. Show analytics and keep learning from the business's own results.

1.3 What was built (working prototype)

Part	What it does
Machine learning model	Calibrated gradient boosting model trained on 9,240 real leads; gives a probability of conversion.
FastAPI backend	All business logic, the agents, the REST API and the webhooks for WhatsApp and email.

SQLite database	Leads, the business profile and the product list.
React dashboard	Onboarding, My Business, lead dashboard, add/edit lead, lead details, follow-ups, analytics, Connect page.
WhatsApp assistant	Through Twilio: welcomes customers, collects their details, answers questions from the product list.
Website form and email	Public enquiry form (also measures time on page and visits); email webhook for forwarding services.
Learning loop	Retrains automatically after every 10 finished leads; a message model learns from the business's own chats.
Test suite	`test_system.py` : messages, scenarios, consistency rules and product-list rules. All pass.

1.4 Key terms in simple words

Term	Meaning
Lead	A person who showed interest (sent a message, filled a form). Not yet a customer.
Conversion	The lead becomes a paying customer. "Converted = 1".
Lead scoring	Giving every lead a number that says how promising it is, so the best are handled first.
Machine learning (ML)	The computer learns patterns from past examples (old leads and whether they bought) instead of us writing every rule.
Classification	An ML task with a yes/no answer: will this lead buy or not?
Probability / chance	A number from 0% to 100%. "70%" means about 70 of 100 similar leads bought.
Feature	One input the model looks at, e.g. lead source or time spent on the website.
API	A set of web addresses the dashboard (or another program) calls to get or save data.
Backend / frontend	Backend = server code and database. Frontend = the website the user sees.
Webhook	A web address that another service (Twilio, an email service) calls when something happens, e.g. a new message.
Agent	A small module with one job (analyse, score, draft a message...). The Orchestrator runs them in order.

2. System architecture

2.1 Architecture diagram

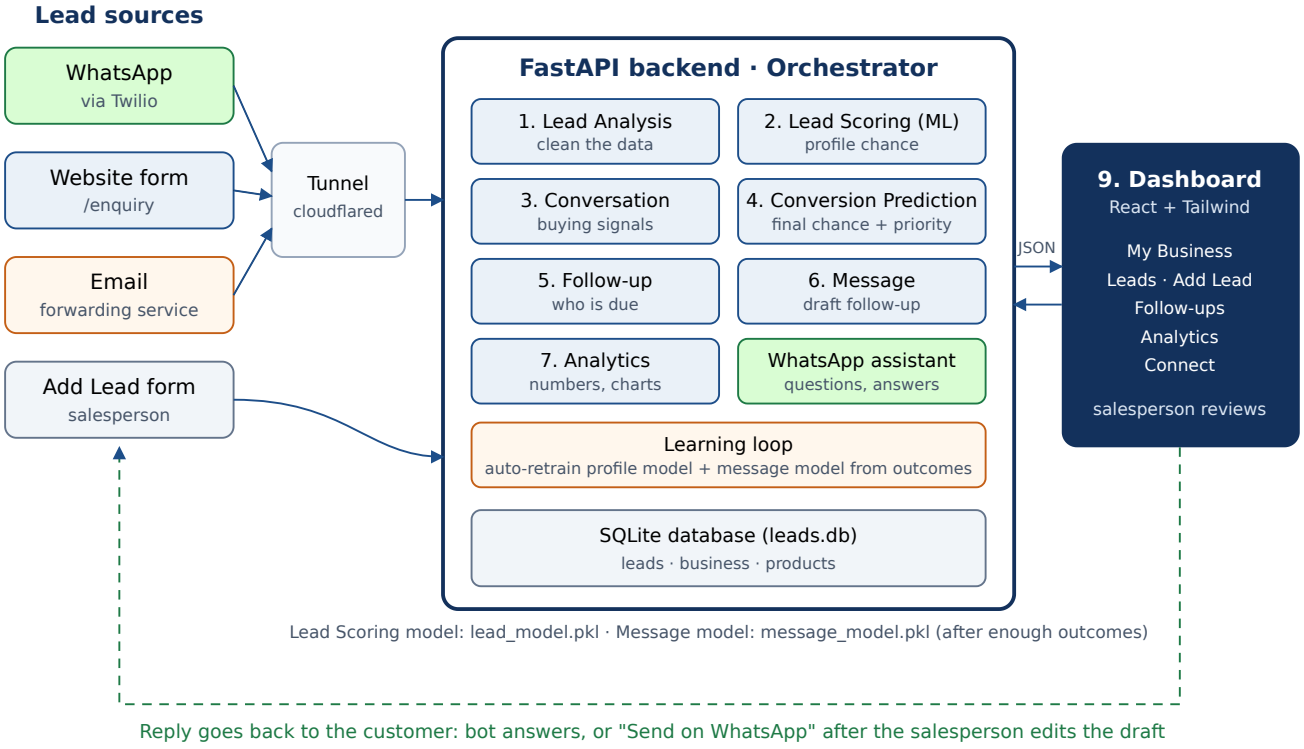


Figure 1 – Lead sources feed the backend; the Orchestrator runs the agents and saves the result; the dashboard shows it.

What happens when a customer writes "Hi, what is the price of the Creta? I want to book this week"

1. Twilio receives the WhatsApp message and calls the backend webhook through the tunnel.
2. The backend checks Twilio's signature, finds or creates the lead for that phone number.
3. **Lead Analysis** cleans the details. **Lead Scoring** asks the ML model for the profile chance.
4. **Conversation** finds "Asked about price" and "Wants to buy soon". **Conversion Prediction** combines them into the final chance and priority.
5. The WhatsApp assistant replies with the Creta's price from the owner's product list.
6. The lead appears on the dashboard as HIGH, with the reason and a drafted follow-up. The salesperson reviews and sends it.

2.2 Technology stack

Layer	Technology	Why it was chosen

Language	Python 3.12	Best libraries for data and ML; simple to read.
Data	pandas, NumPy	Load, clean and transform the CSV dataset.
Machine learning	scikit-learn 1.9	Pipelines, gradient boosting, calibration, evaluation in one library.
Model file	joblib	Saves the trained pipeline (`lead_model.pkl`) so the backend loads it in a second.
Backend	FastAPI + Uvicorn	Fast, automatic input validation (Pydantic) and free test page at `/docs` .
Database	SQLite	One file, no server to install; enough for one business.
Frontend	React 19 + Vite + Tailwind CSS 4	Reusable components; quick development; clean responsive design.
WhatsApp	Twilio WhatsApp Sandbox	Works in minutes without Meta business verification. Meta Cloud API webhook also supported.
Public link	cloudflared quick tunnel	Lets Twilio reach the backend running on a laptop, free, no account.

2.3 Project files

File	Purpose
`train_model.py`	Cleans Leads.csv, trains and compares models, learns cut-offs, saves `lead_model.pkl` .
`agents.py`	Lead Analysis, Lead Scoring, Conversation, Follow-up, Message and Analytics agents; signal weights.
`app.py`	FastAPI app and Orchestrator: lead CRUD, scoring, follow-ups, analytics, retraining, learning status.
`database.py`	All SQLite work: tables, insert, read, update, delete.
`channels.py`	Website form, WhatsApp (Twilio and Meta) and email webhooks; sending on WhatsApp; channel status.
`whatsapp_bot.py`	The WhatsApp assistant: question flow, answers from the product list, typo tolerance.
`business.py`	Business profile and product list API.
`train_message_model.py`	Message model trained on the business's own chats with outcomes.

`test_system.py`	Automatic tests for the whole scoring system.
`public/enquiry.html`	The customer-facing website enquiry form.
`frontend/src/…`	React pages: Onboarding, MyBusiness, Dashboard, AddLead, LeadDetails, FollowUps, Analytics, Connect.
Helpers	`simulate_whatsapp.py` (test without Twilio), `load_demo_chats.py` + `demo_chats.csv` (practice data), `predict_lead.py`.

2.4 The nine modules

Module	Input	Output	Where
Lead Analysis Agent	Raw form / message data	Cleaned lead (trimmed text, Title Case name, lower-case email)	`agents.analyse_lead`
Lead Scoring Agent	Channel, customer type, last activity, visits, time on site	Profile chance + reasons	`LeadScoringAgent`
Conversion Prediction	Profile chance + signals	Final chance, HIGH/MEDIUM/LOW	`apply_signals`
Conversation Agent	Customer's message	Buying-signal tags	`analyse_conversation`
Follow-up Agent	Priority, last contact time	Leads due for follow-up	`needs_follow_up`
Message Agent	Lead, priority, tags	Editable follow-up draft	`draft_message`
Analytics Agent	All leads	Counts, conversion rates by channel	`analytics`
Dashboard	API data	Pages for the sales team and owner	React frontend
Orchestrator	Every request	Runs the agents in order and saves	`app.py`

3. Phase 1 – Dataset and machine learning model

PHASE 1 · DATA & MODEL

3.1 The dataset

The model is trained on `Leads.csv`, the public **Lead Scoring** dataset of an education company (X Education, available on Kaggle). It has **9,240 real leads, 37 columns**, and the column `Converted` (1 = became a customer). **38.5%** of the leads converted.

> **Honest note**
>
> The data comes from one company that sells online courses. The model learns general lead behaviour (source, activity, engagement), not the exact behaviour of a car showroom or a bakery. That is why the system keeps learning from each business's own outcomes (Phase 7).

3.2 Cleaning and avoiding data leakage

- The value `"Select"` means the person left a dropdown empty, so it is treated as missing.
- **Data leakage** means giving the model information that is only known after the result. Columns such as *Tags*, *Lead Quality*, *Lead Profile* and the *Asymmetrique* scores are filled by the sales team after talking to the lead, so they were removed. With them, accuracy looks falsely high (above 90%) but the model would fail on new leads.
- ID columns carry no meaning and were removed.

3.3 Only the inputs a small business can collect

An early version used 25 columns. Most of them are never filled by a real enquiry form, so in real use they were always empty and pulled every score down (many leads showed 26%). The final model uses the **6 columns** the system can fill:

Dataset column	Filled from
Lead Source	Channel (WhatsApp, Instagram, Website, Referral...)
Lead Origin	Worked out from the channel
What is your current occupation	Customer type (Student, Working Professional, Business Owner...)
Last Activity	Last activity (Chatted with us, Had a phone call, Opened our email...)

TotalVisits	Website visits (measured by the enquiry form)
Total Time Spent on Website	Time on website (measured by the enquiry form)

Our form values are mapped to the dataset's values, e.g. WhatsApp/Instagram → "Olark Chat" (chat), Referral → "Reference", "Chatted with us" → "Email Opened" (the closest engaged activity). Rare occupations (Businessman, Housewife) are grouped as "Other" so the model has enough examples. For chat and referral channels an empty "time on website" means the person never visited, so it is set to 0, as in the data.

3.4 Model, preprocessing and training

- **Pipeline:** numbers → missing values filled with the median; a "never visited" flag; categories → one-hot encoding (rare values grouped).
- **Candidates:** Gradient Boosting (HistGradientBoostingClassifier) and Random Forest. Gradient boosting is kept unless Random Forest is better by more than 0.01 F1.
- **Monotonic rule:** more time on the website or more visits can never lower the chance. This stops strange results such as "visited longer, scored lower".
- **Calibration** (sigmoid, 5-fold): the output becomes a true probability; "60%" really means about 60 of 100 such leads buy.
- **Unknown fields:** if customer type or last activity is empty, the model's answer is averaged over the usual values instead of treating "empty" as a bad sign.
- **Fixed test set:** 1,848 leads (20%) kept aside in `test_set.csv` and never used for training, so results are honest and comparable between versions.

3.5 Priority cut-offs learned from data

The thresholds are not guessed. They are learned on a validation split during training:

- **MEDIUM from 40%**: the point with the best F1-score (best balance of finding buyers and avoiding false alarms).
- **HIGH from 56%**: the lowest chance where about 80% of the leads really bought.

3.6 Results on the fixed test set (1,848 leads)

Measure	Value	Meaning
Accuracy	**81.5%**	Correct yes/no answers out of all leads.
Recall	**83.7%**	Of the people who really bought, 83.7% were marked MEDIUM or HIGH.
Precision	**72.6%**	Of the leads marked MEDIUM or HIGH, 72.6% really bought.
F1-score	**0.778**	Balance of precision and recall.

HIGH leads that bought	**79.5%**	When the system says HIGH, it is right about 4 times out of 5.
Brier score	**0.128**	Calibration error (lower is better; 0.25 = guessing).

What the model finds most important (permutation importance)

Time spent on the website matters most, then whether the lead received our SMS/WhatsApp, the customer's occupation (working professionals convert more), and the lead origin.

4. Phase 2 – Backend and database

PHASE 2 · BACKEND

4.1 The Orchestrator flow for a new lead

```
POST /leads  →  Lead Analysis (clean)  →  Lead Scoring (ML profile chance)
             →  Conversation (signals) → Conversion Prediction (final chance, priority,
reasons)
             →  save in SQLite  →  Message Agent (draft)  →  reply to the dashboard
```

4.2 API endpoints

Endpoint	Purpose
`POST /leads`	Add a lead (validated, scored, saved).
`GET /leads` , `GET /leads/{id}`	List leads (HIGH first, then by score) / one lead.
`PUT /leads/{id}`	Edit a lead; it is scored again.
`PATCH /leads/{id}/status`	New / Contacted / Converted / Not Converted.
`PATCH /leads/{id}/priority`	Salesperson sets the priority (or back to automatic).
`DELETE /leads/{id}`	Remove a lead.
`GET /leads/{id}/message`	Drafted follow-up message.
`POST /leads/{id}/whatsapp`	Send the (edited) message on WhatsApp through Twilio.
`GET /followups`	Leads due for follow-up.
`GET /analytics` , `GET /learning`	Dashboard numbers; learning status.
`POST /retrain` , `POST /rescore`	Retrain with outcomes; score every lead again.
`GET/PUT /business` , `GET/PUT /products`	Business profile and product list.
`GET /enquiry` , `POST /public/enquiry`	Website form and its submission.
`POST /webhook/twilio`	Incoming WhatsApp messages from Twilio.
`GET/POST /webhook/whatsapp`	Meta WhatsApp Cloud API (verify + messages).
`POST /webhook/email`	Incoming emails from a forwarding service.
`GET /connections` , `GET /options`	Channel status; dropdown values for the forms.

4.3 Input validation

Pydantic checks every request before any code runs: name 2–60 characters, phone 10–13 digits, a real email format, channel and customer type only from the allowed list, website visits 0–300 and time 0–3000 (the range seen in the training data; the model cannot judge values far outside it). Wrong input returns HTTP 422 with a message for each field, which the dashboard shows under the right box.

4.4 Database tables

Table	Columns
leads	id, name, phone, email, channel, interested_in, customer_type, last_activity, customer_message, intent_tags, website_visits, time_on_website, **score** (final chance), **profile_score** (model only), priority, priority_override, reasons, status, chat_step, created_at, last_contacted_at
business	name, business_type, owner_name, whatsapp_number, working_hours, delivery_area, delivery_charge, advance_notice, special_instructions
products	id, name, price, details, available

New columns are added automatically when the program starts, so existing leads are never lost when the system is updated.

5. Phase 3 – Reading the customer's message

PHASE 3 · CONVERSATION & FINAL SCORE

5.1 Buying signals

The Conversation Agent looks for keyword groups (whole words, so "rate" does not match "separate"). Common typos are corrected first ("specfications" → specifications, "pirce" → price). Each signal changes the **odds** of buying by a weight:

Signal	Example words	Odds ×
Wants to buy soon	buy, book, today, this week, join, register	2.5
Asked for a call or demo	call me, demo, visit, appointment	2.0
Asked about price	price, cost, how much, fees, EMI, quotation	1.6
Asked about product details	specifications, features, colour, available, warranty, test drive	1.6
Asked about discounts	discount, offer, deal	1.2
Comparing options / Has questions	compare, vs / what, how, ?	1.0 (shown only)
Just browsing	just checking, will see later, let you know	0.5
Not in our product list	asks for something the business does not sell	0.5
Only said hi	hi, hello, ok, thanks (nothing else)	0.35
Not interested	not interested, don't call, wrong number	0.1 and always LOW

5.2 How the final chance is calculated

```
odds        = chance / (1 − chance)
final odds  = profile odds × weight of every signal   (all signals together at most × 5)
final chance = final odds / (1 + final odds)
```

> **Worked example**
>
> Business Owner on WhatsApp: profile chance **66%** → odds 1.94. Asked about price and specifications: 1.94 × 1.6 × 1.6 = 4.97 → final chance **83%**, HIGH. The lead page shows "Profile 66% +17% from the message = 83%". The same lead saying only "Hi" gets 1.94 × 0.35 = 0.68 → 40%.

The priority is decided from the final chance with the learned cut-offs, so the percentage and the HIGH/MEDIUM/LOW badge always agree. A salesperson who knows the customer can set the priority manually; it stays until they choose "Automatic".

5.3 Why the weights are rules and not a trained model (yet)

A message model needs messages *together with* the result (bought or not). We tested a public dataset of 100,000 sales conversations (DeepMostInnovations/saas-sales-conversations, synthetic, software sales) on 40,000 of them:

- From a customer's first 1–3 messages, TF-IDF + Logistic Regression reached AUC **0.58–0.61** (0.5 = guessing). Only full conversations reached 0.85, which is not useful at the first message.
- Its strongest words were product names from that dataset, and our signals had almost no relation to its outcomes (70% of its customers ask about price).

So we did not use it. The weights follow sales common sense and are replaced by a **message model trained on the business's own chats** once enough outcomes exist (Phase 7). This decision, backed by the measurement, is itself a result of the project.

5.4 Follow-up and Message agents

- **Follow-up:** HIGH leads are due after 1 day, MEDIUM after 3, LOW after 7 (new HIGH leads at once). Closed leads are never due.
- **Message:** a template for each priority plus one line per signal, e.g. "I'll share the full price details with you right away." For "Not interested" it sends a polite closing message instead. The salesperson always edits and sends it; nothing is sent to a customer from the dashboard without a person clicking.

6. Phase 4 – Website dashboard

PHASE 4 · FRONTEND

Page	What the user does there
Onboarding (first visit)	Four steps: business profile → products & services → policies → review & finish. Clear error messages; cannot continue with missing required fields.
My Business	Read-only overview with Edit buttons for contact & hours, products, policies. Business name and type are fixed (Reset onboarding to change).
Dashboard	All leads, HIGH first: score bar, priority badge (✎ if set by hand), status, intent tags, search and status filter.
Add Lead	Manual entry with the same validation as the backend; shows the score and reasons immediately.
Lead details	Score breakdown (profile + message), reasons, customer's messages, editable draft, Send on WhatsApp, status and priority buttons, edit and delete.
Follow-ups	Leads due now with their drafted message.
Analytics	Leads by channel, priority and status; conversion rate by channel (finished leads only); learning status; Retrain and Re-score buttons.
Connect (own navbar button)	Status and lead count for WhatsApp, Website and Email; the exact addresses to paste; website embed code; send-a-test-email form.

All numbers on the dashboard come from the backend API (no hard-coded demo values). The layout works on a phone (menus wrap, tables scroll).

7. Phase 5 – Channels: WhatsApp, website and email

PHASE 5 · LEAD SOURCES

7.1 Website enquiry form

`/enquiry` is a public page a business links from its website or embeds with an iframe. It validates the input in the browser, measures time on the page and number of visits (stored only in the visitor's browser), and creates a **Website** lead with real values for the two most important model inputs. The customer only sees a thank-you message; the score stays with the sales team.

7.2 WhatsApp through Twilio

1. The backend runs on the laptop; `cloudflared tunnel --url http://localhost:8000` gives a public https link.
2. In the Twilio Sandbox, "When a message comes in" is set to `https://<link>/webhook/twilio` (POST). The plain link also works.
3. Each phone sends the sandbox's *join* code once (membership lasts 72 hours).
4. Every message is checked with Twilio's **X-Twilio-Signature** (HMAC-SHA1 with the auth token), so fake requests are rejected.
5. The reply is returned as TwiML; "Send on WhatsApp" uses Twilio's Messages API.

> **Real limits found during testing**
>
> The free trial has a daily message limit (error 63038). Replies only reach numbers that joined the sandbox and wrote in the last 24 hours (WhatsApp rule). Error 11210 meant Twilio still had an old tunnel link; 11200 meant the backend was not running. The quick-tunnel link changes on every restart.

The Meta WhatsApp Cloud API webhook (verify token + X-Hub-Signature-256) is also implemented for production use.

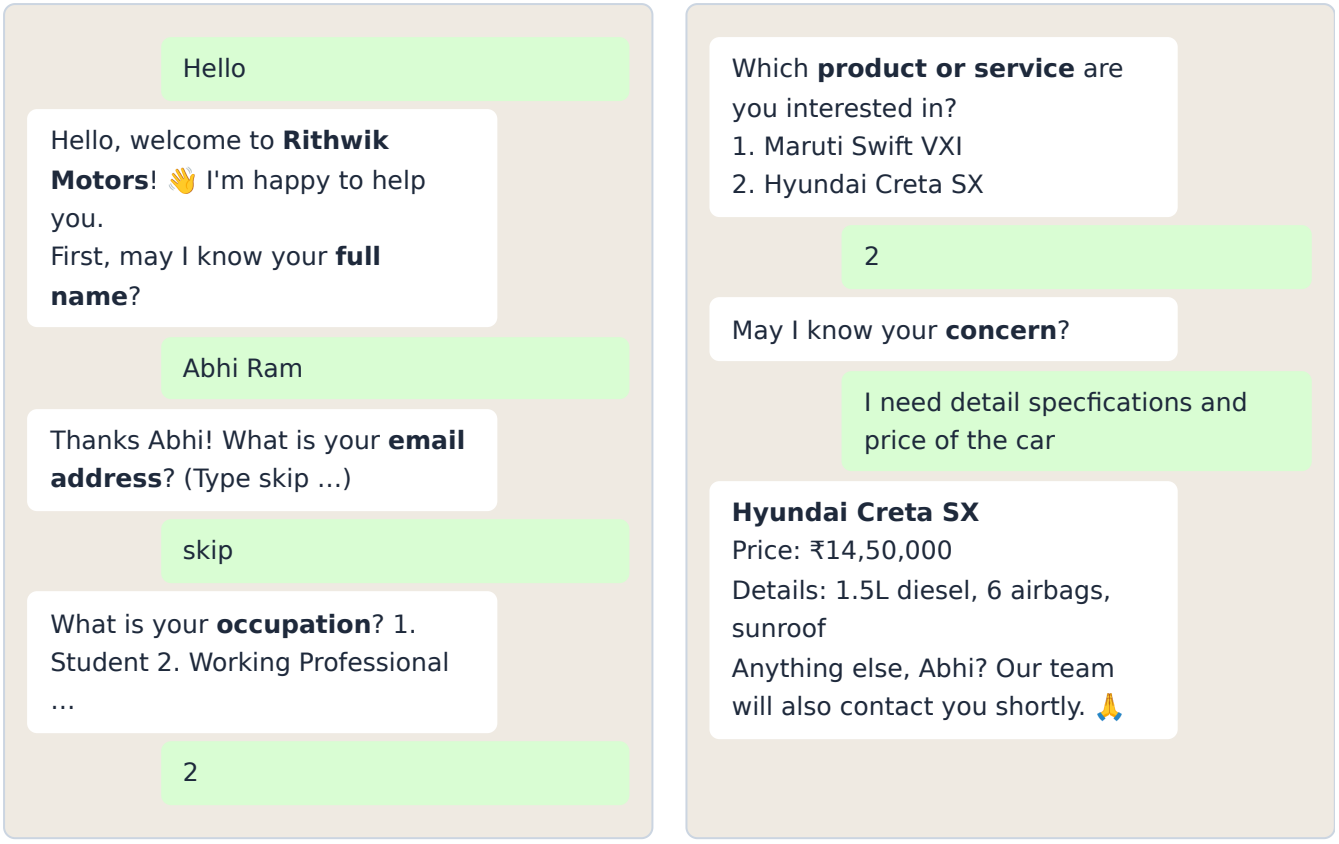
7.3 Email

An email-forwarding service (CloudMailin, Zapier, Make) posts each customer email to `/webhook/email` as JSON or form fields `from`, `subject`, `text`. "Priya Sharma <priya@gmail.com>" is split into name and address; a new sender becomes an Email lead and the same sender writing again updates it. An optional token protects the address.

8. Phase 6 – The WhatsApp assistant

PHASE 6 · ASSISTANT

The assistant collects the details the model needs, one question at a time, and answers questions from the owner's product list. Every answer is saved into the lead and the lead is scored again.



- **Prices are hidden in lists** and given only when asked, in Indian format (₹14,50,000).
- It answers **details, availability, delivery** (area, charge, notice) and **working hours**; it understands common typos.
- "non veg thali price" picks *Non-Veg Thali*, not *Veg Thali* (the product matching the most words wins).
- Asked for something not sold ("do you have bikes?"): "Sorry, we don't have that. We offer: ..." and the lead's odds are halved.
- A question in the middle of the questions is answered first, then the question is asked again. "Hi" is never taken as a name.
- A returning customer saying "Hi" gets "Hello Abhi, welcome back...". Questions it cannot answer (discounts, custom requests) are left for the sales team.

9. Phase 7 – Learning from results

PHASE 7 · LEARNING LOOP

1. The salesperson marks leads **Converted** or **Not Converted**. These are the real labels.
2. After every **10 new finished leads**, retraining starts by itself in the background (also a Retrain button). A lock stops two trainings running together.
3. **Profile model:** the outcomes are added to Leads.csv and the model is retrained. The new model is kept **only if it scores better** on the same fixed test set; the old one is backed up as `lead_model_previous.pkl`.
4. **Message model:** TF-IDF + Logistic Regression on the customers' messages vs the outcome. It is used only with at least 15 converted and 15 not-converted chats **and** a cross-validated AUC of at least 0.60. From then on, the message's effect comes from what this business's customers really wrote.
5. Every lead is re-scored with the new models.

> **Practice data**
>
> `demo_chats.csv` has 32 practice chats (16 bought, 16 did not) so the loop can be demonstrated. On them the message model reached a cross-validated AUC of 0.72. These outcomes are invented for practice and are removed with `python load_demo_chats.py --remove` before real use.

10. Phase 8 – Testing

PHASE 8 · TESTING

10.1 Automatic system test (`python test_system.py`)

Section	What is checked	Result
1. Conversation Agent	Over 50 customer messages and the signals they must and must not give	All correct
2. Lead scenarios	26 typical leads and the priority they should get	26 / 26
3. Consistency rules	More time or visits never lowers the score; phone call/SMS ≥ no activity; working professional ≥ unemployed	All hold
4. Product list	Asking for products not sold lowers the score; normal questions do not	6 / 6

10.2 Other tests performed

- Full WhatsApp conversations through the Twilio webhook with valid and invalid signatures (invalid → 403).
- Twilio signature function checked against Twilio's published example value.
- Sending on WhatsApp with Twilio's API mocked, including the sandbox error message.
- Email webhook with JSON, form data, "Name <address>" senders, and an invalid sender (422).
- Automatic retraining after 10 outcomes; priority override surviving re-scoring.
- Browser tests (Playwright) of onboarding with validation, editing, Connect page and phone-size layout.
- Live tests with real phones on the Twilio sandbox.

10.3 Sample test cases

Input	Expected	Result
Phone "12345"	422: "Phone must be 10-13 digits"	Pass
Website visits 5000	422: at most 300	Pass
Referral, phone call, 15 min on site	HIGH	Pass (94%)
Website only, nothing else	LOW	Pass (22%)
Strong lead + "Not interested, thanks"	LOW	Pass

Working professional on WhatsApp, only "Hi"	LOW or MEDIUM	Pass (52%)
"Can I book a test drive tomorrow?"	Wants to buy soon + details, HIGH	Pass
Car showroom, "do you have bikes?"	Not in our product list	Pass

11. How to run the project

Three terminals, from the `lead-model` folder (Mac):

```
# Terminal 1 - backend
cd ~/developer/lead-model
source .venv/bin/activate
uvicorn app:app --reload          # wait for "Application startup complete"

# Terminal 2 - public link (only for WhatsApp / email)
cloudflared tunnel --url http://localhost:8000

# Terminal 3 - dashboard
cd ~/developer/lead-model/frontend
npm run dev                       # open http://localhost:5173
```

- Twilio Sandbox → "When a message comes in" → `https://<tunnel-link>/webhook/twilio`, POST, Save.
- `.env` holds TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_WHATSAPP_FROM (never shared or committed).
- Test without WhatsApp: `python simulate_whatsapp.py "Abhi Ram" 919281043214 "Hello"`
- Retrain the base model: `python train_model.py Leads.csv`. Run the tests: `python test_system.py`. API test page: `http://127.0.0.1:8000/docs`.

12. Demonstration script (about 8 minutes)

1. **Problem (1 min):** enquiries from many channels, no time, hot buyers missed.
2. **Setup (1 min):** show the onboarding steps or My Business with the product list.
3. **WhatsApp (2 min):** from a phone send "Hello", answer the questions, ask "price and specifications?". Show the bot's answer, then the lead on the dashboard with its score, tags and reasons.
4. **Contrast (1 min):** a second phone says only "Hi" or asks "do you have bikes?" → lower score. Explain the odds formula with the breakdown line.
5. **Follow-up (1 min):** open the lead, edit the draft, Send on WhatsApp; status becomes Contacted.
6. **Other channels (1 min):** Connect page; submit the website form and a test email.
7. **Learning (1 min):** Analytics → learning status; explain retraining after 10 outcomes; show the model results table.

> **Before the demo**
>
> Rejoin the sandbox from each phone, do not use up the daily Twilio limit on the same day, keep `simulate_whatsapp.py` ready as a backup, and run `caffeinate -d` so the laptop does not sleep.

13. Limitations and future work

Limitations

- The profile model is trained on one education company's leads; its percentages are estimates for other businesses until their own outcomes are added.
- Signal weights are expert rules until the message model has enough real chats.
- Keyword reading misses some phrasing, sarcasm and languages other than English.
- The Twilio sandbox needs a join code, has a daily limit, and is for testing only; the quick tunnel link changes and needs the laptop running.
- SQLite and a single business per installation; no login or user roles yet.
- Prototype, not tested with real customers at scale.

Future work

- Deploy the backend on a cloud server with a fixed domain and a managed database (PostgreSQL); add login.
- Register the business's own WhatsApp number (Twilio WhatsApp Senders or Meta Cloud API).
- Support Telugu, Hindi and Hinglish messages.
- Optional LLM drafting with a rule-based fallback and cost limits.
- Instagram DMs through Meta's API; scheduled reminders by WhatsApp or email.

14. Viva questions and answers

14.1 Project and problem

Q1. What is your project in one line?

A system that collects customer enquiries from WhatsApp, a website and email, predicts how likely each one is to buy, tells the sales team whom to contact first and why, and drafts the follow-up.

Q2. Why do small businesses need it?

They get many enquiries but have little time and no data team. Without priorities, hot buyers wait while the team chats with casual visitors, and follow-ups are forgotten. Big CRM tools are too costly and complex for them.

Q3. What is a lead and what is conversion?

A lead is a person who showed interest. Conversion means the lead became a paying customer. In the dataset it is the column Converted (1 or 0).

Q4. How is your project different from a normal CRM?

A CRM only stores contacts. Ours predicts the chance of buying with ML, reads the customer's message for buying signals, explains every score, answers customers automatically from the product list, and learns from the business's own results.

Q5. Who uses the system?

The owner sets up the business and products; the salesperson works with the lead list, follow-ups and messages. Customers only see WhatsApp, the website form or email.

14.2 Data and machine learning

Q6. Which dataset did you use and why?

The public Lead Scoring dataset (X Education, Kaggle): 9,240 real leads with 37 columns and the real outcome. It is one of the few public datasets with real lead behaviour and results.

Q7. What is data leakage? How did you avoid it?

Using information that is only known after the result. Columns like Tags and Lead Quality are written by the sales team after talking to the lead, so they reveal the answer. We removed them; otherwise accuracy looks above 90% but fails on new leads.

Q8. Why only 6 input columns?

Those are the only ones a real enquiry can fill. With 25 columns, most were always empty in real use and pulled every score down. Training on what we can actually collect gives honest scores.

Q9. Which algorithm did you use and why?

Gradient boosting (HistGradientBoostingClassifier), compared with Random Forest. Boosting builds many small decision trees, each fixing the mistakes of the previous ones. It handles mixed data well and supports monotonic constraints.

Q10. What is a monotonic constraint?

A rule that an input can only push the score in one direction. We set it so more time on the website or more visits can never lower the chance, which matches common sense and removes strange results.

Q11. What is calibration?

Adjusting the model's outputs so they are true probabilities. After calibration, among leads scored 60%, about 60% really bought. We used sigmoid calibration with 5-fold cross-validation.

Q12. How did you measure accuracy?

On a fixed test set of 1,848 leads never used in training: accuracy 81.5%, recall 83.7%, precision 72.6%, F1 0.778, and 79.5% of HIGH leads really bought.

Q13. Explain precision, recall and F1 simply.

Precision: of the leads we flagged, how many really bought. Recall: of the real buyers, how many we flagged. F1 combines both into one number; it is high only when both are good.

Q14. Why is recall important here?

Missing a real buyer loses a sale; contacting a lead who does not buy only costs a call. So we prefer to catch most buyers.

Q15. How are the HIGH / MEDIUM / LOW thresholds chosen?

Learned from data during training: MEDIUM from about 40% (best F1), HIGH from about 56% (where about 80% of leads really bought).

Q16. What if a field is empty?

For customer type and last activity, the model's answer is averaged over the usual values (weighted by how common they are), so "unknown" is not treated as a bad sign.

Q17. Which inputs matter most?

Time spent on the website, receiving our SMS/WhatsApp, occupation (working professionals convert more) and lead origin, measured with permutation importance.

Q18. What happens when a new outcome arrives?

It is saved as a label. After every 10 new outcomes, the system retrains automatically; the new model is kept only if it scores better on the fixed test set, otherwise the old one stays.

14.3 Scoring and messages

Q19. How does the customer's message change the score?

The Conversation Agent finds buying signals. Each multiplies the odds by a weight (buy soon × 2.5, price × 1.6, only "Hi" × 0.35, not interested × 0.1). Example: 66% with a price and a specs question becomes 83%.

Q20. Why odds and not just adding points?

Odds keep the result between 0% and 100% and make a signal matter more for a lead in the middle than for one already at 95%. It is the same idea logistic regression uses.

Q21. Why didn't you train the message part on a dataset?

We tried: 40,000 public sales conversations gave AUC only 0.58–0.61 from the first messages (0.5 is guessing), and they were synthetic software-sales chats. So we use explainable weights and a message model that learns from the business's own chats once enough outcomes exist.

Q22. What is AUC?

The probability that the model ranks a random buyer above a random non-buyer. 0.5 is guessing, 1.0 is perfect.

Q23. What is TF-IDF?

A way to turn text into numbers: a word gets a high value if it appears in this message but is rare across all messages. Logistic regression then learns a weight for each word.

Q24. Why does asking for a product you don't sell lower the score?

The business cannot sell that customer what they want, so a sale is less likely. The odds are halved and the tag "Not in our product list" is shown.

Q25. Can the salesperson disagree with the system?

Yes. They can set the priority themselves; it is marked ✎ and stays until they switch back to Automatic. The model's percentage is still shown.

Q26. Did you use an LLM like ChatGPT?

No. Messages are drafted from templates plus signal lines, and the bot answers from the product list. This is free, fast, works offline and never invents prices. An LLM with a fallback is possible future work.

14.4 Backend, database and dashboard

Q27. Why FastAPI?

It is fast, validates input automatically with Pydantic, and gives a free test page at /docs.

Q28. What does the Orchestrator do?

It runs the agents in the right order for every request (analyse → score → read message → final chance → save → draft) and returns the result.

Q29. Why SQLite?

One file, no server, enough for one small business. For many users we would move to PostgreSQL; the code keeps all database work in database.py so the change is in one place.

Q30. How do you avoid losing data when the system changes?

New columns are added with ALTER TABLE at start-up; an old table is renamed as a backup, never deleted. Old models are backed up before being replaced.

Q31. How does the frontend talk to the backend?

With fetch calls to the REST API in api.js. Errors from the backend (422) are shown under the correct form field.

Q32. What is CORS and why do you need it?

A browser rule that blocks a page from calling a server on another address. The dashboard (port 5173) calls the backend (port 8000), so the backend allows it.

14.5 WhatsApp, website and email

Q33. How do WhatsApp messages reach your system?

Twilio receives them and calls our webhook address. Because the backend runs on a laptop, a cloudflared tunnel gives it a public https link.

Q34. What is a webhook?

A URL that another service calls when an event happens. Twilio calls /webhook/twilio for every new WhatsApp message.

Q35. How do you know a request really came from Twilio?

Twilio signs each request with our auth token (HMAC-SHA1 in the X-Twilio-Signature header). We recompute it; if it does not match, we reject with 403.

Q36. Why Twilio and not Meta directly?

The Twilio sandbox works in minutes without business verification. In production a business would connect its own number through Meta's Cloud API (also implemented) or Twilio WhatsApp Senders.

Q37. What does the bot do and not do?

It welcomes the customer, collects name, email, occupation and product, and answers price, details, availability, delivery and timing questions from the owner's list. It does not negotiate discounts; those go to the sales team.

Q38. What is the 24-hour rule?

A business can send free-form WhatsApp messages only within 24 hours of the customer's last message. After that, only pre-approved templates are allowed.

Q39. How does the website form help the model?

It measures time on the page and visits, the two engagement inputs the model values most, instead of leaving them empty.

Q40. How does email work?

A forwarding service posts each email to /webhook/email. The sender's name and address, subject and text become an Email lead, scored like any other.

14.6 Testing, security and limitations

Q41. How did you test the system?

test_system.py checks over 50 messages, 26 lead scenarios, consistency rules and product-list rules; plus webhook tests with real and fake signatures, browser tests of the dashboard, and live WhatsApp tests.

Q42. How do you keep secrets safe?

Twilio keys are in a .env file read by the backend only; never in the frontend, code, report or GitHub.

Q43. How do you protect customer data?

We collect only what is needed for scoring and contact, validate all input, store it locally, and do not share it. A real deployment would add login and HTTPS hosting.

Q44. What are the main limitations?

The model is trained on one company's data; message weights are rules until real outcomes exist; English keywords only; sandbox limits; no login yet.

Q45. Is the prediction always right?

No. It is a probability, not a promise: about 4 of 5 HIGH leads bought in testing. It helps decide whom to call first; the salesperson still decides.

Q46. How would you deploy it for a real business?

Backend on a cloud server with a fixed domain, PostgreSQL database, the business's own WhatsApp number, login for staff, and regular backups.

Q47. What did you learn?

Honest evaluation matters more than a high number: removing leakage, using only collectable inputs, testing a dataset before trusting it, and building a system that learns from real results.

15. Glossary

Term	Meaning
Accuracy	Share of correct predictions.
AUC	How well a model ranks positives above negatives (0.5 guessing, 1.0 perfect).
Calibration	Making predicted probabilities match real frequencies.
Cross-validation	Training and testing several times on different splits to get a reliable score.
Data leakage	Using information that would not be available at prediction time.
Endpoint	One API address with a method, e.g. POST /leads.
HMAC signature	A code computed with a secret key that proves who sent a message.
JSON	Text format for data sent between frontend and backend.
Odds	chance / (1 − chance). 75% = odds 3.
One-hot encoding	Turning a category into 0/1 columns the model can read.
Pipeline	Preprocessing and model saved together, so new data is treated exactly like training data.
Pydantic	Library that checks request data against rules.
REST API	Web API using GET, POST, PUT, PATCH, DELETE.
Tunnel	A service that gives a laptop's server a temporary public address.
TwiML	Twilio's XML format for replies.
Webhook	An address another service calls when something happens.